

FULL STACK

ASSIGNMENT – 1

IMPLEMENT INHERITANCE TYPES USING JAVASCRIPT

Student Name: K. Hansikha

Roll Number: A24126552264

Department: CSM (AI&ML)

Section: B

Subject Name: FULL STACK

Assignment No.: 1

Date: 29-08-2026

Academic Year: 2026

Technology Used: JavaScript with Node.js

1. Objective

To implement and demonstrate different types of inheritance in JavaScript using Node.js and understand how classes and objects reuse methods and behavior.

2. Introduction

Inheritance is an object-oriented programming concept in which a child class acquires methods and behavior from a parent class. JavaScript supports class inheritance through the **extends** keyword. Multiple inheritance is demonstrated using a mixin because JavaScript does not directly allow a class to extend two classes.

3. Types Implemented

- Single Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance
- Multiple Inheritance using Mixins
- Hybrid Inheritance

4. Execution

The complete program is stored in one file named **inheritance.js** and executed using **node inheritance.js**.

5. Complete Source Code — inheritance.js

The following is the complete Node.js source code used for the assignment.

```
// Assignment 1: Inheritance Types using JavaScript
// Run using Node.js

console.log("=====");
console.log(" INHERITANCE TYPES USING JAVASCRIPT");
console.log("=====\\n");

// 1. SINGLE INHERITANCE
console.log("1. SINGLE INHERITANCE");
console.log("-----");

class Person {
  introduce() {
    console.log("Person: I can introduce myself.");
  }
}

class Student extends Person {
  study() {
    console.log("Student: I can study.");
  }
}

const student = new Student();
student.introduce();
student.study();

// 2. MULTILEVEL INHERITANCE
console.log("\\n2. MULTILEVEL INHERITANCE");
console.log("-----");

class Vehicle {
  start() {
    console.log("Vehicle: Engine started.");
  }
}

class Car extends Vehicle {
  drive() {
    console.log("Car: I can drive.");
  }
}

class ElectricCar extends Car {
  charge() {
    console.log("Electric Car: Battery is charging.");
  }
}

const electricCar = new ElectricCar();
electricCar.start();
electricCar.drive();
electricCar.charge();

// 3. HIERARCHICAL INHERITANCE
console.log("\\n3. HIERARCHICAL INHERITANCE");
console.log("-----");

class Shape {
  display() {
    console.log("Shape: This is a geometric shape.");
  }
}

class Circle extends Shape {
  drawCircle() {
    console.log("Circle: Drawing a circle.");
  }
}

class Rectangle extends Shape {
  drawRectangle() {
    console.log("Rectangle: Drawing a rectangle.");
  }
}

const circle = new Circle();
const rectangle = new Rectangle();

circle.display();
```

```

circle.drawCircle();

rectangle.display();
rectangle.drawRectangle();

// 4. MULTIPLE INHERITANCE USING MIXINS
console.log("\n4. MULTIPLE INHERITANCE USING MIXINS");
console.log("-----");

class Coder {
  coding() {
    console.log("Coder: I can write programs.");
  }
}

const Communicator = {
  communicate() {
    console.log("Communicator: I can communicate.");
  }
};

class Developer extends Coder {
  develop() {
    console.log("Developer: I can develop applications.");
  }
}

Object.assign(Developer.prototype, Communicator);

const developer = new Developer();

developer.coding();
developer.communicate();
developer.develop();

// 5. HYBRID INHERITANCE
console.log("\n5. HYBRID INHERITANCE");
console.log("-----");

class Employee {
  work() {
    console.log("Employee: I can perform my work.");
  }
}

class Programmer extends Employee {
  program() {
    console.log("Programmer: I can write code.");
  }
}

class Manager extends Employee {
  manage() {
    console.log("Manager: I can manage a team.");
  }
}

const LeaderSkills = {
  lead() {
    console.log("Leader: I can lead a team.");
  }
};

class TechnicalManager extends Programmer {
  manageProjects() {
    console.log("Technical Manager: I can manage technical projects.");
  }
}

Object.assign(TechnicalManager.prototype, LeaderSkills);

const technicalManager = new TechnicalManager();

technicalManager.work();
technicalManager.program();
technicalManager.manageProjects();
technicalManager.lead();

console.log("\n=====");
console.log(" ALL INHERITANCE TYPES COMPLETED");
console.log("=====");

```

6. Explanation

Single Inheritance: One child class inherits from one parent class.

Multilevel Inheritance: Classes form an inheritance chain, with a derived class becoming the base for another class.

Hierarchical Inheritance: Multiple child classes inherit from the same parent class.

Multiple Inheritance using Mixins: A mixin adds behavior from another source to a class, providing a practical way to combine behaviors.

Hybrid Inheritance: A combination of inheritance techniques, demonstrated with class inheritance and a mixin.

7. Output

The screenshot below shows the successful Node.js execution of all five inheritance types.

```
c:\Users\dadhi\OneDrive\Desktop\clg_programs\full-stack\assignments\assignment-1\codes>node inheritance.js
=====
      INHERITANCE TYPES USING JAVASCRIPT
=====

1. SINGLE INHERITANCE
-----
Person: I can introduce myself.
Student: I can study.

2. MULTILEVEL INHERITANCE
-----
Vehicle: Engine started.
Car: I can drive.
Electric Car: Battery is charging.

3. HIERARCHICAL INHERITANCE
-----
Shape: This is a geometric shape.
Circle: Drawing a circle.
Shape: This is a geometric shape.
Rectangle: Drawing a rectangle.

4. MULTIPLE INHERITANCE USING MIXINS
-----
Coder: I can write programs.
Communicator: I can communicate.
Developer: I can develop applications.

5. HYBRID INHERITANCE
-----
Employee: I can perform my work.
Programmer: I can write code.
Technical Manager: I can manage technical projects.
Leader: I can lead a team.
```

Result: All five inheritance types were implemented and executed successfully using JavaScript with Node.js.

Conclusion: The assignment demonstrates inheritance concepts through JavaScript classes, the extends keyword, and mixins.